

SENA — ADSO 2026

Clase 33

Diseño Técnico del Proyecto

API de Supabase — Base de Datos PostgreSQL

Texticode — Sistema de Gestión Textil

Aprendices

Kevin Stevan Wagner
Kevin Andres Castro
Andres Felipe Lancheros
Camilo Andres Tibambre

Instructor

Ing. Edwin Marín

Fecha

Mayo 2026

Programa

ADSO 2026

1. Arquitectura 3 Capas

Texticode implementa una arquitectura REST desacoplada en 3 capas. Supabase actúa como proveedor de base de datos PostgreSQL gestionada en la nube, reemplazando la instancia MySQL local para permitir acceso colaborativo desde cualquier equipo del equipo ADSO.

FRONTEND Vue.js 3 · GestionOrdenes.vue, GestionClientes.vue · Axios HTTP · VITE_API_KEY en .env
↓ HTTP / REST (x-api-key)
BACKEND Node.js + Express · routes/ordenes.js · middleware apiKey.js · Validación de negocio
↓ SQL queries (pg.Pool — Supabase Session Pooler puerto 6543)
BASE DE DATOS Supabase PostgreSQL · us-west-1 · Project ID: oizxmymoagvasyculdma · SSL habilitado

FLUJO DE DATOS — CREAR UNA ORDEN DE PRODUCCIÓN

Capa	Acción
Frontend	GestionOrdenes.vue → axios.post('/api/ordenes', body, { headers: { 'x-api-key': KEY } })
Backend Auth	middleware apiKey.js verifica x-api-key contra API_KEY_EFICIENCIA en .env
Backend Business	ordenes.js valida campos obligatorios: Id_Cliente, Id_Material, Descripcion, Cantidad, Fecha_Limite
Persistence	pg.Pool ejecuta INSERT INTO orden_produccion ... RETURNING Id_Orden
Database	Supabase PostgreSQL inserta con garantías ACID — Session Pooler puerto 6543
Response	201 Created + { mensaje: 'Orden creada', Id_Orden: 148 }

2. Contrato Técnico API

Los 4 contratos de la API de Órdenes documentados con evidencia real de pruebas en Thunder Client.

2.1 GET /api/ordenes

GET /api/ordenes

Lista todas las órdenes de producción con cliente y material mediante JOIN. Requiere API Key.

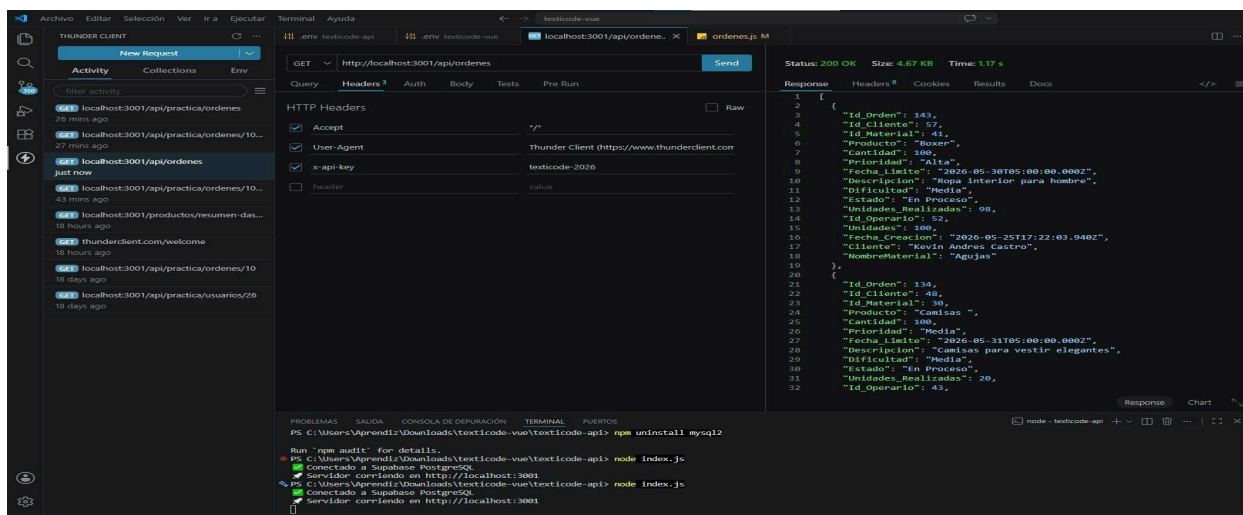
REQUEST

```
GET /api/ordenes HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026
```

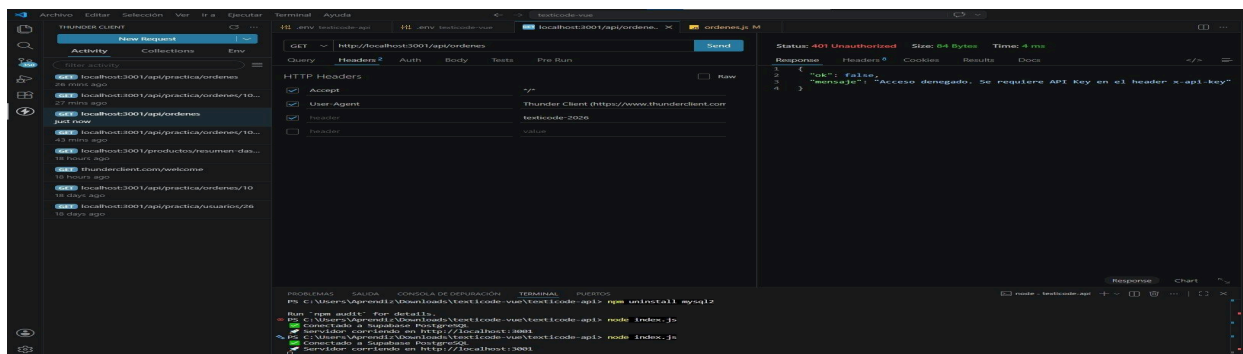
```
// Respuesta: array de órdenes con campos Cliente y NombreMaterial incluidos via JOIN
// Ordenadas por Fecha_Limite ASC
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — API Key correcta → 200 OK



Prueba 2 — Sin header x-api-key → 401 Unauthorized



2.2 POST /api/ordenes

POST /api/ordenes

Crea una nueva orden de producción. Devuelve el Id_Orden generado por Supabase.

REQUEST

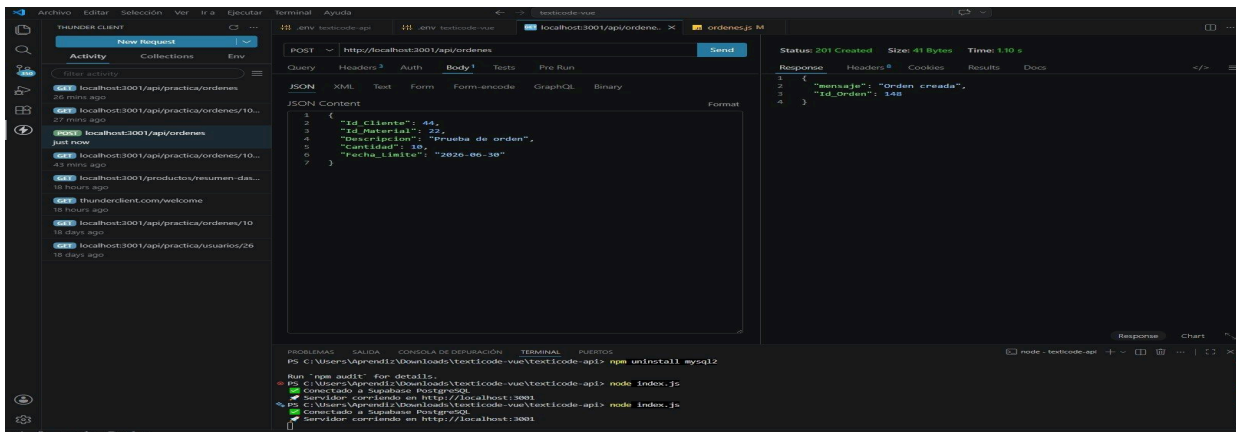
```
POST /api/ordenes HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026
Content-Type: application/json

{
  "Id_Cliente": 44,
  "Id_Material": 22,
  "Descripcion": "Prueba de orden",
  "Cantidad": 10,
  "Fecha_Limite": "2026-06-30"
}
```

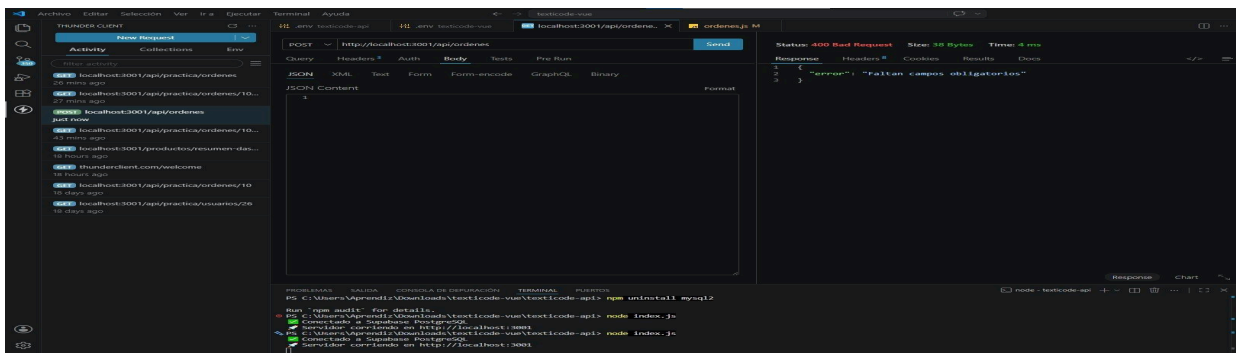
// Campos requeridos: Id_Cliente, Id_Material, Descripcion, Cantidad, Fecha_Limite
// Campos opcionales: Id_Operario, Producto, Prioridad, Estado, Dificultad

EVIDENCIA THUNDER CLIENT

Prueba 1 — Body completo y correcto → 201 Created



Prueba 2 — Body vacío (campos obligatorios faltantes) → 400 Bad Request



2.3 PUT /api/ordenes/:id

PUT /api/ordenes/:id

Actualiza todos los campos de una orden existente. Devuelve 404 si el Id_Orden no existe.

REQUEST

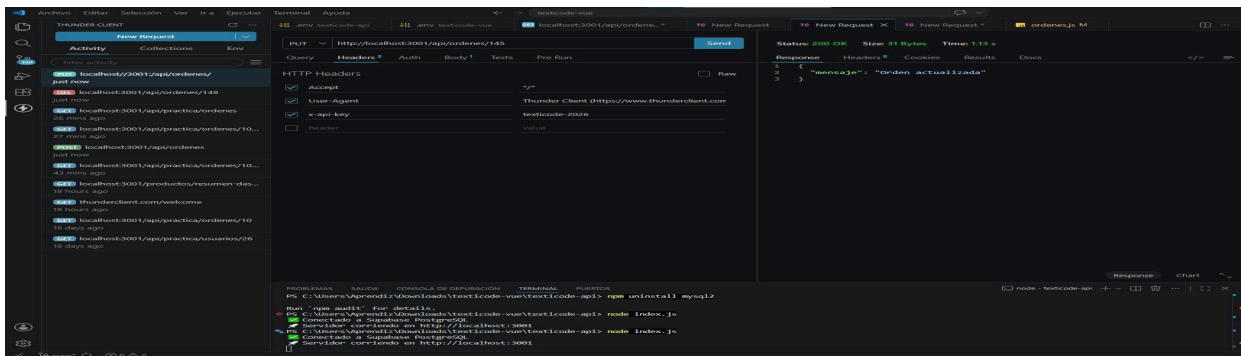
```
PUT /api/ordenes/145 HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026
Content-Type: application/json

{
  "Id_Cliente": 44, "Id Material": 22, "Id Operario": 52,
  "Producto": "Boxer", "Descripcion": "Prueba actualización",
  "Cantidad": 10, "Prioridad": "Alta", "Fecha_Limite": "2026-06-30",
  "Estado": "En Proceso", "Unidades": 10, "Unidades_Realizadas": 5,
  "Dificultad": "Media"
}

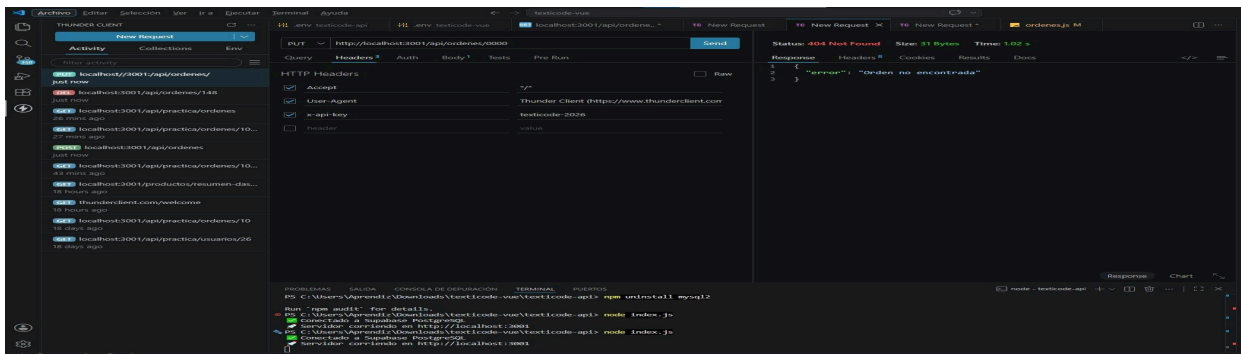
// :id → Id_Orden (entero positivo)
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Orden existente (id: 145) → 200 OK



Prueba 2 — Orden inexistente (id: 0000) → 404 Not Found



2.4 DELETE /api/ordenes/:id

DELETE /api/ordenes/:id

Elimina permanentemente una orden de producción por su Id_Orden.

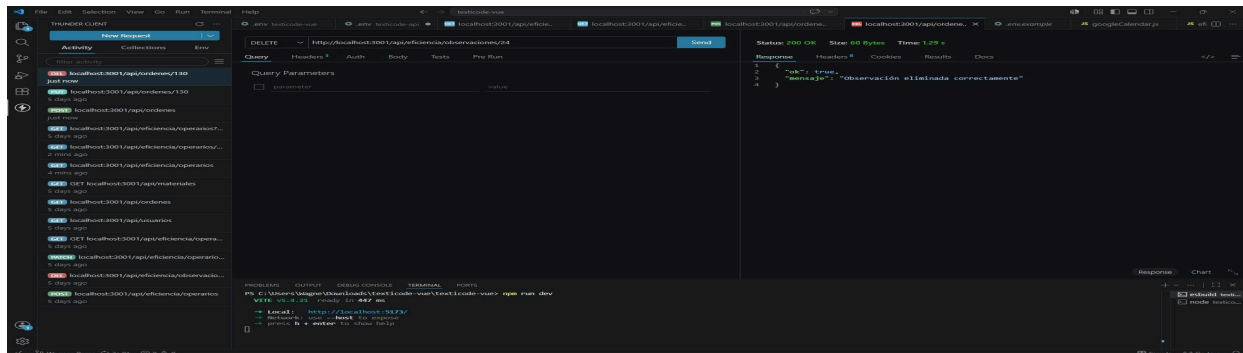
REQUEST

```
DELETE /api/ordenes/148 HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026

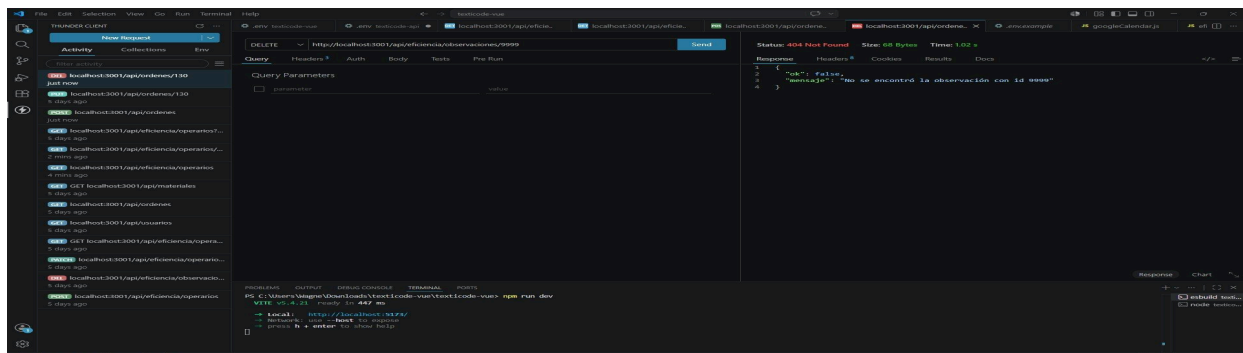
// :id → Id_Orden (entero positivo)
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Orden existente (id: 148) → 200 OK



Prueba 2 — Orden inexistente (id: 9999) → 404 Not Found



3. Diagrama Entidad-Relación

Texticode opera sobre 6 tablas en Supabase PostgreSQL. La tabla central es orden_produccion que vincula clientes, materiales, operarios y comprobantes.

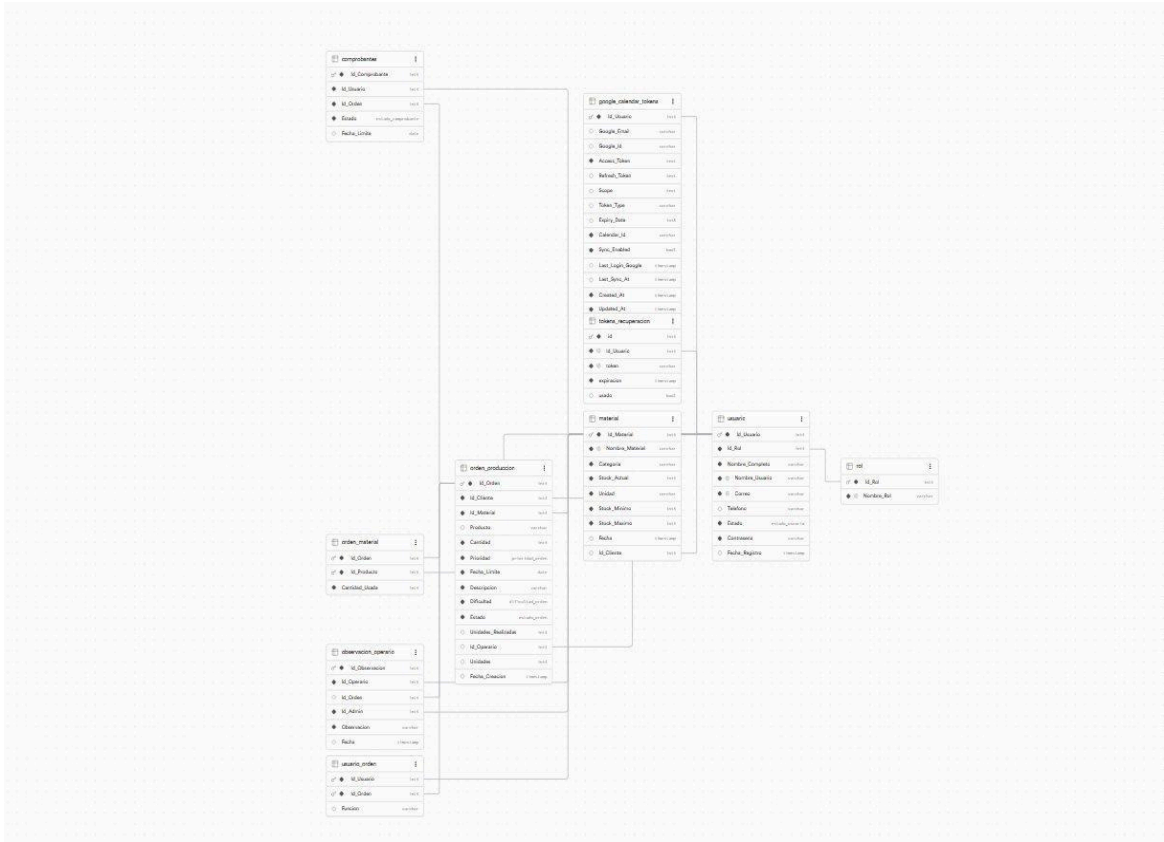


Tabla	Campos
usuario	Id_Usuario (PK) · Nombre_Completo · Nombre_Usuario · Correo (UNIQUE) · Telefono · Estado · Id_Rol (FK)
rol	Id_Rol (PK) · Nombre_Rol (admin operario cliente)
orden_produccion	Id_Orden (PK) · Id_Cliente (FK) · Id_Material (FK) · Id_Operario (FK) · Producto · Descripcion · Cantidad · Prioridad · Dificultad · Estado · Unidades · Unidades_Realizadas · Fecha_Limite · Fecha_Creacion
material	Id_Material (PK) · Nombre_Material · Categoria · Stock_Actual · Stock_Minimo · Stock_Maximo · Unidad · Id_Cliente (FK)
observacion_operario	Id_Observacion (PK) · Id_Operario (FK) · Id_Orden (FK) · Id_Admin (FK) · Observacion · Fecha
comprobantes	Id_Comprobante (PK) · Id_Usuario (FK) · Id_Orden (FK) · Estado · Fecha_Limite

RELACIONES CLAVE

Origen	Relación	Destino	Descripción
usuario	1 → N	orden_produccion	Un cliente tiene múltiples órdenes asignadas
usuario	1 → N	observacion_operario	Un operario acumula múltiples observaciones
orden_produccion	1 → N	observacion_operario	Una orden puede tener múltiples observaciones
material	1 → N	orden_produccion	Un material se usa en múltiples órdenes
orden_produccion	1 → N	comprobantes	Una orden genera uno o más comprobantes
rol	1 → N	usuario	Un rol agrupa múltiples usuarios

4. Validaciones y Manejo de Errores

Todos los endpoints de /api/ordenes comparten el middleware apiKey.js y aplican validación de campos obligatorios antes de ejecutar queries a Supabase.

Escenario	Validación en código	HTTP
API Key ausente	<code>!req.headers['x-api-key']</code>	401 Unauthorized
API Key incorrecta	<code>key !== process.env.API_KEY_EFICIENCIA</code>	403 Forbidden
Campos obligatorios faltantes	<code>!Id_Cliente !Id_Material !Descripcion !Cantidad !Fecha_Limite</code>	400 Bad Request
Orden no encontrada (PUT/DELETE)	<code>rowCount === 0</code>	404 Not Found
Estado inválido	<code>!['En Proceso','Completada','Pausado'].includes(Estado)</code>	400 Bad Request
ID no numérico	<code>isNaN(req.params.id)</code>	400 Bad Request
Error interno de BD	<code>catch(err) → log interno</code>	500 Server Error

CONEXIÓN A SUPABASE (DB.JS)

```
// texticode-api/db.js
import pg from 'pg'
import dotenv from 'dotenv'
dotenv.config()
const { Pool } = pg
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false } // requerido por Supabase
})
export default pool
```